Redondeo y ajuste por mínimos cuadrados ponderados

Daniel Martín Guerrero

Laboratorio de Mecánica y Ondas, Grado en Física
Universidad de Salamanca

octubre de 2026



Índice

1. Objetivo
2. El redondeo
3. El ajuste ponderado
4. Resultado

Objetivo

Cuando hacemos una gráfica con ajuste, el código se encarga de dos tareas:

- Calcular la recta que mejor describe los datos, teniendo en cuenta lo fiable que es cada punto.
- Presentar el resultado con el número correcto de cifras significativas.

Vamos a recorrer el código línea a línea con los números reales del ajuste de la velocidad del sonido (caso 1 del informe): v frente a $x = 1 - f_{\mathrm{E}}/f_{\mathrm{R}}$.

Criterio de redondeo

Criterio del informe:

- la incertidumbre se da con *una* cifra significativa;
- si esa cifra es un 1, se conservan *dos*;
- el valor se redondea a la misma posición decimal.

Sin redondear	1.ª cifra	Resultado
A: $0{,}353\,17 \pm 0{,}011\,8$	1	$0{,}353 \pm 0{,}012$
B: $244{,}66 \pm 88{,}6$	8	$(2{,}4 \pm 0{,}9) \times 10^2$

Los ejemplos A y B nos acompañarán en toda esta sección.

La función redondea

```python
def redondea(valor, incertidumbre):
    exp = math.floor(math.log10(abs(incertidumbre)))
    primera = int(abs(incertidumbre) / 10**exp)
    decimales = -(exp - 1) if primera == 1 else -exp
    v, u = round(valor, decimales), round(incertidumbre, decimales)
    d = max(decimales, 0)
    return f"{v:.{d}f}".replace(".", ","), f"{u:.{d}f}".replace(".", ",")
```

Recibe el valor y su incertidumbre sin redondear y devuelve dos textos listos para el informe. Los pasos son:

1. localizar la primera cifra significativa;
2. ver si es un 1;
3. decidir dónde cortar;
4. redondear y escribir con coma decimal.

Paso 1: posición de la primera cifra

```
exp = math.floor(math.log10(abs(incertidumbre)))
```

El logaritmo decimal responde a la pregunta: *¿a qué número hay que elevar 10 para obtener este?*

Número	Potencia	$\log_{10}$
100	10^2	2
10	10^1	1
1	10^0	0
0,1	10^{-1}	-1
0,01	10^{-2}	-2

Si el número cae entre dos potencias, su logaritmo cae entre los dos enteros:

- $\log_{10}(88{,}6) \approx 1{,}95$;
- $\log_{10}(0{,}0118) \approx -1{,}93$.

`abs(...)` garantiza que el número sea positivo.

Paso 1: el papel de `floor`

`math.floor` redondea siempre hacia abajo y conserva la parte entera, que indica la posición de la primera cifra significativa:

	$\log_{10}$	`floor`	Posición
A: 0,0118	$-1{,}93$	-2	centésimas (10^{-2})
B: 88,6	$1{,}95$	1	decenas (10^{1})

- En los negativos, «hacia abajo» significa *alejándose del cero*: $-1{,}93$ pasa a -2, no a -1, y así debe ser, porque el 1 de 0,0118 está en las centésimas.
- A ojo es fácil ver dónde empieza un número; el logaritmo es la forma matemática de que el ordenador lo averigüe para cualquier valor.

Paso 2: valor de la primera cifra

```
primera = int(abs(incertidumbre) / 10**exp)
```

Dividir entre 10^{exp} desplaza la coma hasta dejar la primera cifra delante, e `int(...)` corta todo lo que hay detrás:

$$\text{A: } \frac{0{,}0118}{10^{-2}} = 1{,}18 \;\to\; 1, \qquad \text{B: } \frac{88{,}6}{10^{1}} = 8{,}86 \;\to\; 8. \tag{1}$$

Ya sabemos dónde está la primera cifra (exp) y cuál es (`primera`): con eso basta para aplicar el criterio.

Paso 3: posición de corte

```
decimales = -(exp - 1) if primera == 1 else -exp
```

- Si la primera cifra es un 1, se conservan dos cifras: se corta una posición más a la derecha, `-(exp - 1)`.
- En otro caso se conserva una: `-exp`.

	exp	primera	decimales
A	-2	1	$-(-2-1)=3$ (milésimas)
B	1	8	-1 (decenas)

Un número de decimales negativo indica que se redondea a la izquierda de la coma.

Paso 4: redondeo y escritura

```
v, u = round(valor, decimales), round(incertidumbre, decimales)
d = max(decimales, 0)
return f"{v:.{d}f}".replace(".", ","), f"{u:.{d}f}".replace(".", ",")
```

- `round` redondea valor e incertidumbre a la misma posición.
- `max(decimales, 0)`: si se redondea a las decenas, no hay decimales que escribir.
- El formato escribe exactamente `d` decimales y `replace` cambia el punto por la coma.

	Entrada	Devuelve
A	0,353 17; 0,011 8	`0,353` y `0,012`
B	244,66; 88,6	`240` y `90`, es decir, $(2{,}4 \pm 0{,}9) \times 10^2$

La función coma y dos detalles

```python
def coma(x, d):
    return f"{x:.{d}f}".replace(".", ",")
```

Solo da formato, sin aplicar el criterio de redondeo; se usa para números auxiliares como $t_{95,\nu}$ o χ^2/ν.

Empates: Python lleva los 5 exactos a la cifra par (`round(0.125, 2)` da 0,12), como indica la regla de las diapositivas de teoría de errores.

Caso límite: $u = 0{,}0196$ empieza por 1 y conserva dos cifras, 0,020; tras redondear empieza por 2, pero mantiene las dos. Es poco frecuente y, si aparece, se corrige a mano.

Motivación

Cada punto es la media de seis lecturas, con $u^2 = s^2/n + \Delta^2/12$, y la abscisa $x = 1 - f_\mathrm{E}/f_\mathrm{R}$ hereda la incertidumbre de las frecuencias:

Serie	$u(y)$ (m/s)	$a\,u(x)$ (m/s)
Máxima, acercamiento	0,0026	0,030
Máxima, alejamiento	0,0012	0,184
Media, acercamiento	0,0009	0,011
Media, alejamiento	0,0008	0,015
Mínima, acercamiento	0,0004	0,025
Mínima, alejamiento	0,0007	0,008

La incertidumbre horizontal, trasladada a la vertical, es entre 10 y 150 veces mayor: el ajuste no puede ignorar la x ni tratar igual todos los puntos.

La función ajuste_ponderado (I)

```
def ajuste_ponderado(x, ux, y, uy, modelo="libre"):
    x, ux, y, uy = (np.asarray(t, float) for t in (x, ux, y, uy))
    n = len(x)
    a = np.polyfit(x, y, 1)[0] if modelo == "libre" else (x @ y) / (x @ x)
    for _ in range(500):
        w = 1 / (uy**2 + a**2 * ux**2)
        if modelo == "libre":
            W = w.sum()
            m = lambda z: (w * z).sum() / W
            a_nueva = (m(x * y) - m(x) * m(y)) / (m(x * x) - m(x) ** 2)
        else:
            a_nueva = (w * x * y).sum() / (w * x * x).sum()
        if abs(a_nueva - a) <= 1e-12 * max(1.0, abs(a)):
            a = a_nueva
            break
        a = a_nueva
```

La función ajuste_ponderado (II)

```
ue = np.sqrt(uy**2 + a**2 * ux**2)
w = 1 / ue**2
if modelo == "libre":
    W = w.sum()
    m = lambda z: (w * z).sum() / W
    b = m(y) - a * m(x)
    ua = 1 / math.sqrt(W) / math.sqrt(m(x * x) - m(x) ** 2)
    ub = ua * math.sqrt(m(x * x))
    nu = n - 2
else:
    b, ub, nu = 0.0, 0.0, n - 1
    ua = 1 / math.sqrt((w * x * x).sum())
chi2 = float((w * (y - a * x - b) ** 2).sum())
factor = max(1.0, math.sqrt(chi2 / nu)) if nu > 0 else 1.0
t95 = float(stats.t.ppf(0.975, nu)) if nu > 0 else float("nan")
```

Preparación y pendiente inicial

```
x, ux, y, uy = (np.asarray(t, float) for t in (x, ux, y, uy))
n = len(x)
a = np.polyfit(x, y, 1)[0] if modelo == "libre" else (x @ y) / (x @ x)
```

- `np.asarray` convierte las listas en *arrays*, con los que se opera punto a punto: `x * y` multiplica cada x_i por su y_i.
- `n = len(x)` cuenta los puntos: $n = 6$.
- La iteración necesita una pendiente de partida:
  - modelo libre: el ajuste sin pesos de `np.polyfit`;
  - recta por el origen: $a = \sum x_i y_i \big/ \sum x_i^2$.

En el caso 1 la pendiente de partida es $a_0 = 82{,}4$ m/s: el ajuste básico, arrastrado por el punto anómalo.

Pesos y varianza efectiva

```
w = 1 / (uy**2 + a**2 * ux**2)
```

Peso de cada punto:

$$w_i = \frac{1}{u_i^2}, \qquad u_i^2 = u^2(y_i) + a^2\, u^2(x_i). \tag{2}$$

- Un punto con el doble de incertidumbre pesa cuatro veces menos: los puntos precisos mandan en la recta.
- ¿Por qué a^2? Un desplazamiento horizontal δx equivale, sobre una recta de pendiente a, a uno vertical $a\,\delta x$; sumando ambas contribuciones como varianzas independientes se obtiene u_i^2.
- Para calcular w_i hace falta a, que es justo lo que se busca: por eso hay que iterar.

La media ponderada

```
W = w.sum()
m = lambda z: (w * z).sum() / W
```

`lambda` define una función en una línea; z es solo el nombre de su argumento. `m` calcula la media ponderada:

$$\langle z \rangle = \frac{\sum_i w_i\, z_i}{\sum_i w_i}. \tag{3}$$

Llamada	Calcula	Caso 1
`m(x)`	$\langle x \rangle$	$3{,}27 \times 10^{-4}$
`m(y)`	$\langle y \rangle$	$-0{,}0214$
`m(x*x)`	$\langle x^2 \rangle$	$6{,}49 \times 10^{-7}$
`m(x*y)`	$\langle xy \rangle$	$1{,}26 \times 10^{-4}$

Ejemplo: con pesos $(1, 1, 2)$ y valores $(10, 20, 40)$, $\langle z \rangle = 110/4 = 27{,}5$, mayor que la media aritmética (23,3), porque el 40 pesa el doble.

La pendiente ponderada

```
a_nueva = (m(x * y) - m(x) * m(y)) / (m(x * x) - m(x) ** 2)
```

Mínimos cuadrados ponderados: minimizando $\chi^2 = \sum_i w_i \left(y_i - a x_i - b\right)^2$ respecto de a y b,

$$a = \frac{\langle xy \rangle - \langle x \rangle \langle y \rangle}{\langle x^2 \rangle - \langle x \rangle^2}, \qquad b = \langle y \rangle - a \langle x \rangle. \tag{4}$$

- El numerador es una «covarianza» ponderada entre x e y, y el denominador, una «varianza» ponderada de x.
- Con todos los pesos iguales se recuperan las fórmulas básicas de las notas de incertidumbres.
- Para la recta por el origen:

  `a_nueva = (w*x*y).sum() / (w*x*x).sum()`.

Criterio de convergencia

```
if abs(a_nueva - a) <= 1e-12 * max(1.0, abs(a)):
    a = a_nueva
    break
a = a_nueva
```

Si la pendiente nueva apenas difiere de la anterior (una billonésima relativa), el cálculo ha convergido y se sale con `break`; si no, se guarda y se da otra vuelta. `range(500)` es solo un tope de seguridad.

Vuelta	a (m/s)
Inicio	82,4
1	244,279
2	244,657
3	244,657

El salto de 82 a 244 m/s en la primera vuelta es el efecto de los pesos: el punto anómalo deja de arrastrar la recta.

Pesos finales

```
ue = np.sqrt(uy**2 + a**2 * ux**2)
w = 1 / ue**2
```

Serie	u_i (m/s)	Peso
Mínima, alejamiento	0,008	51 %
Media, acercamiento	0,011	26 %
Media, alejamiento	0,015	14 %
Mínima, acercamiento	0,025	5 %
Máxima, acercamiento	0,030	3,5 %
Máxima, alejamiento	0,184	0,1 %

Con la pendiente ya convergida se recalculan la incertidumbre efectiva de cada punto y su peso definitivo.

El punto anómalo prácticamente no cuenta; las series a velocidad media y mínima, las más precisas, deciden la recta.

Ordenada e incertidumbres de los parámetros

```
b = m(y) - a * m(x)
ua = 1 / math.sqrt(W) / math.sqrt(m(x * x) - m(x) ** 2)
ub = ua * math.sqrt(m(x * x))
nu = n - 2
```

Incertidumbres de los parámetros:

$$u(a) = \frac{1}{\sqrt{\sum_i w_i}\sqrt{\langle x^2\rangle - \langle x\rangle^2}}, \qquad u(b) = u(a)\sqrt{\langle x^2\rangle}. \tag{5}$$

- $b = -0{,}1014$ m/s: la recta pasa por el centro de gravedad ponderado.
- $u(a) = 7{,}67$ m/s, $u(b) = 0{,}0062$ m/s y $\nu = 6 - 2 = 4$.

Bondad del ajuste: χ^2

```
chi2 = float((w * (y - a * x - b) ** 2).sum())
factor = max(1.0, math.sqrt(chi2 / nu)) if nu > 0 else 1.0
```

$$\chi^2 = \sum_i \left(\frac{y_i - a x_i - b}{u_i} \right)^2 . \tag{6}$$

- Si las barras de error están bien estimadas, $\chi^2 \approx \nu$.
- En el caso 1, $\chi^2 = 69{,}3$ frente a $\nu = 4$: la dispersión supera lo que indican las barras y $u(a) = 7{,}67$ m/s es demasiado optimista.
- Se corrige con $\sqrt{69{,}3/4} = 4{,}16$; `max(1.0, ...)` impide que la incertidumbre disminuya.

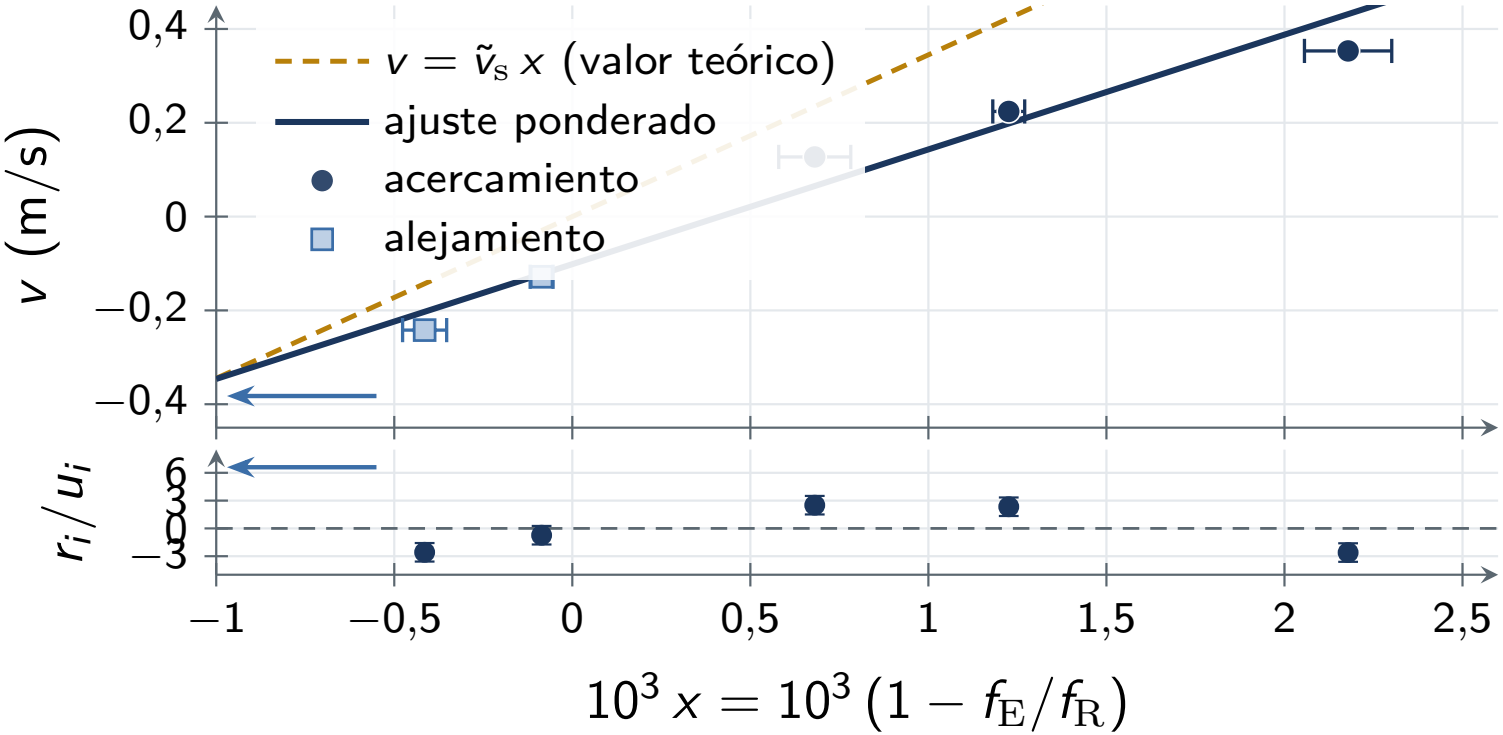
La t de Student y el resultado

```
t95 = float(stats.t.ppf(0.975, nu)) if nu > 0 else float("nan")
return dict(a=a, b=b, ua=ua * factor, Ua=t95 * ua * factor, ...)
```

`stats.t.ppf(0.975, 4)` deja un 2,5 % en cada cola, es decir, un intervalo central del 95 %: $t_{95,4} = 2{,}78$, mayor que el 1,96 de la gaussiana, porque con pocos puntos la propia incertidumbre está mal conocida.

Clave	Contenido	Caso 1
a	pendiente	244,657 m/s
ua	$u(a)$ corregida	$7{,}67 \times 4{,}16 = 31{,}9$ m/s
Ua	$t_{95,4}\, u(a)$	$2{,}78 \times 31{,}9 = 88{,}6$ m/s
b, Ub	ordenada	$-0{,}101 \pm 0{,}071$ m/s

Del código al resultado



$$v_\mathrm{S} = (2{,}4 \pm 0{,}9) \times 10^2 \text{ m/s}, \qquad b = (-0{,}10 \pm 0{,}07) \text{ m/s}$$

Incertidumbres ampliadas al 95 %, con $\nu = 4$.

Resumen

Redondeo:

1. el logaritmo y `floor` localizan la primera cifra;
2. dividir y truncar indica cuál es;
3. si es un 1 se conservan dos cifras; si no, una;
4. valor e incertidumbre se cortan en la misma posición.

Ajuste ponderado:

1. cada punto pesa $1/u_i^2$, con la varianza efectiva;
2. se itera hasta que la pendiente no cambia;
3. b, $u(a)$ y $u(b)$ se calculan con medias ponderadas;
4. χ^2/ν indica si hay que corregir las incertidumbres;
5. la t de Student da el intervalo al 95 %.